

Large Language Models (LLMs): A Complete Guide

Architecture, Training, Capabilities, and Applications

A structured reference document covering transformer architecture, training methods, prompting, retrieval-augmented generation, fine-tuning, evaluation, agents, applications, and limitations.

Prepared June 2026

Table of Contents

1. Introduction to Large Language Models
2. The Transformer Architecture
3. How LLMs Are Trained
4. Scale, Parameters, and Emergent Behavior
5. Context Windows and Memory
6. Prompting and In-Context Learning
7. Retrieval-Augmented Generation (RAG)
8. Fine-Tuning Approaches
9. Inference: How LLMs Generate Text
10. Multimodal LLMs
11. Limitations and Challenges
12. Evaluating LLMs
13. AI Agents and Tool Use
14. Real-World Applications of LLMs
15. Cost, Deployment, and Security Considerations
16. The Future of LLMs
17. Glossary of Key Terms

1. Introduction to Large Language Models

Large Language Models, commonly abbreviated as LLMs, are a class of artificial intelligence systems trained on massive collections of text to understand and generate human language. Unlike earlier natural language processing systems that relied on hand-crafted rules or small statistical models, LLMs learn the structure, grammar, facts, and reasoning patterns of language directly from data, using deep neural networks with billions or even trillions of parameters.

The term 'large' refers both to the size of the model itself (the number of trainable parameters) and the size of the training corpus (often hundreds of billions to trillions of words drawn from books, websites, code repositories, and other text sources). This scale is what allows modern LLMs to perform tasks they were never explicitly trained for, a phenomenon often called emergent ability.

Examples of well-known LLM families include the GPT series, the Claude family, Gemini, Llama, and Mistral, among many others. While these models differ in architecture details, training data, and fine-tuning approach, they share a common foundation: the transformer architecture.

It is useful to think of an LLM as a very sophisticated pattern-completion engine. At its core, the model has learned a probability distribution over what word or token is likely to come next, given everything that came before it in a piece of text. Repeating this prediction step over and over, one token at a time, is enough to produce essays, working computer programs, poetry, translations, and step-by-step explanations of difficult concepts. The surprising part is not the mechanism itself, which is conceptually simple, but how far this simple mechanism goes once the model and its training data are scaled up to enormous size.

1.1 Why LLMs Matter

LLMs have changed how people interact with computers. Instead of writing rigid commands or queries, users can describe what they want in natural language and receive a coherent, often highly useful, response. This has unlocked applications across writing assistance, coding, customer support, education, research, translation, and creative work.

- Natural interaction: Users communicate in plain language rather than learning specialized syntax.
- Generalization: A single model can perform translation, summarization, coding, and reasoning tasks.
- Knowledge synthesis: LLMs combine information learned during training to answer novel questions.
- Productivity: Drafting, editing, and analysis tasks that took hours can now take minutes.

1.2 A Brief Historical Timeline

- 2013-2014: Word embeddings (Word2Vec, GloVe) capture word meaning as vectors.
- 2014-2017: Recurrent neural networks (RNNs) and LSTMs dominate sequence modeling.

- 2017: The paper 'Attention Is All You Need' introduces the Transformer architecture.
- 2018: BERT demonstrates the power of large-scale pretraining for language understanding.
- 2018-2020: The GPT series (GPT, GPT-2, GPT-3) scales up decoder-only transformers.
- 2022: ChatGPT brings conversational LLMs to mainstream public attention.
- 2023-2026: Multimodal, longer-context, and more efficient models become standard, with reasoning-focused and agentic models emerging as a major new direction.

1.3 How People Access LLMs

There are several common ways individuals and organizations interact with LLMs in practice. Chat interfaces, such as a web app or mobile app, let a person type messages and receive responses directly, similar to a conversation. Application programming interfaces (APIs) allow developers to send requests to a model programmatically and receive responses that their own software can process, which is the foundation for most LLM-powered products. Many providers also offer software development kits (SDKs), command-line tools, and integrations inside existing productivity software such as spreadsheets, word processors, browsers, and email clients, lowering the barrier to embedding LLM capability directly into everyday workflows.

1.4 Open-Weight vs. Closed/Proprietary Models

LLMs are often categorized by how their trained weights are distributed. Closed or proprietary models are accessible only through an API or product controlled by the company that trained them; the underlying weights are not released publicly. Open-weight models make the trained parameters available for anyone to download, inspect, modify, and run on their own hardware, subject to a license. Open-weight availability enables greater transparency, customization, and local or offline deployment, while closed models often provide access to the most capable systems along with managed infrastructure, support, and continuous improvements without requiring the user to manage hardware themselves.

2. The Transformer Architecture

Almost every modern LLM is built on the transformer architecture, introduced in 2017. The key innovation of the transformer is the self-attention mechanism, which allows the model to weigh the relevance of every other token in a sequence when processing a given token, regardless of distance between them. This solved a major limitation of earlier recurrent architectures, which struggled to capture long-range dependencies in text.

2.1 Tokens and Tokenization

Before any text can be processed by a model, it must be converted into tokens: small units such as whole words, sub-words, or even individual characters. Most modern LLMs use sub-word tokenization schemes such as Byte-Pair Encoding (BPE) or similar variants. This allows the model to handle rare words, misspellings, and multiple languages by breaking unfamiliar words into smaller known pieces.

As a concrete illustration, a common English word such as 'token' might be represented as a single token, while a rarer or compound word might be split into multiple pieces, such as a root and a suffix. A made-up or highly unusual word, or a word in a language poorly represented in training data, may be broken down into several smaller fragments, sometimes down to individual characters, so that the model can still process it even though it has never seen that exact word before. This is also why token counts do not map directly onto word counts: a rough rule of thumb for English text is that one token corresponds to roughly three quarters of a word on average, though this varies by language and tokenizer.

Each token is mapped to a numerical identifier, and then to a dense vector called an embedding. These embeddings are the starting point for all subsequent computation inside the network.

2.2 Self-Attention

Self-attention computes, for every token, a weighted combination of all other tokens' representations. Each token is projected into three vectors: a Query, a Key, and a Value. The similarity between a token's Query and every other token's Key determines how much attention (weight) is paid to that other token's Value. This lets the model dynamically decide which parts of the input are relevant to understanding each word in context.

Consider the sentence 'The trophy did not fit in the suitcase because it was too big.' Resolving what 'it' refers to requires connecting that pronoun back to either 'trophy' or 'suitcase', depending on context. Self-attention allows the model to directly link the representation of 'it' to the representation of whichever earlier word is most relevant for this particular sentence, without needing to process every word in between sequentially, the way older recurrent models did.

Multi-head attention runs several attention computations in parallel, each potentially learning to focus on different types of relationships (for example, grammatical structure versus topical relevance), and then combines the results.

2.3 Positional Information

Because self-attention has no inherent sense of word order, transformers add positional information to the token embeddings. Early transformers used fixed sinusoidal positional encodings; many modern LLMs use rotary positional embeddings (RoPE) or other learned schemes that generalize better to longer sequences than seen during training.

2.4 Feed-Forward Layers and Layer Stacking

Each transformer block contains a self-attention layer followed by a position-wise feed-forward neural network, with residual connections and layer normalization around both. Dozens or even hundreds of these blocks are stacked on top of each other, allowing the model to build increasingly abstract representations of the input as information passes through deeper layers.

2.5 Encoder, Decoder, and Decoder-Only Models

The original transformer had two parts: an encoder that processes the input and a decoder that generates output, connected by cross-attention. Models like the original Transformer and T5 use this full encoder-decoder design, well suited to tasks like translation. BERT-style models use only the encoder, ideal for understanding tasks like classification. Most modern generative LLMs, including the GPT family and similar chat models, use a decoder-only architecture, where the model predicts the next token based only on the tokens that came before it.

Architecture	Typical Use Case	Example Models
Encoder-only	Text understanding, classification, embeddings	BERT, RoBERTa
Decoder-only	Open-ended text generation, chat, reasoning	GPT family, Claude, Llama
Encoder-decoder	Translation, summarization with structured input/output	T5, original Transformer

3. How LLMs Are Trained

3.1 Pretraining

The first and most resource-intensive stage of building an LLM is pretraining. During pretraining, the model is shown enormous amounts of text and learns to predict the next token in a sequence, given everything that came before it. This objective, called causal language modeling, is deceptively simple but forces the model to implicitly learn grammar, facts, reasoning patterns, and even some limited world knowledge, purely as a side effect of getting better at prediction.

Pretraining typically requires thousands of specialized accelerator chips (GPUs or TPUs) running for weeks or months, consuming very large amounts of electricity and computing budget. The resulting model is often called a base model or foundation model.

3.2 Data Sources and Data Quality

Pretraining data is usually drawn from a mixture of web pages, digitized books, scientific papers, code repositories, encyclopedic content, and curated high-quality corpora. The exact mixture and quality filtering process used by any given lab is closely tied to the resulting model's strengths and weaknesses. Heavy filtering for quality, deduplication, and removal of harmful or low-value content has become a major focus, since data quality often matters as much as data quantity.

Deduplication deserves particular attention: the same or near-identical text often appears many times across the web, and training repeatedly on duplicated content can waste compute and, in some cases, increase the risk that a model memorizes and later reproduces specific passages verbatim rather than learning generalizable patterns. Modern data pipelines therefore invest heavily in near-duplicate detection across enormous datasets before training begins.

Synthetic data, meaning text generated by an existing AI model rather than collected from human sources, has also become an increasingly important ingredient in modern training pipelines, particularly for generating large quantities of high-quality worked examples in domains like mathematics, coding, and structured reasoning where naturally occurring text of sufficient quality and density is comparatively scarce.

3.3 Supervised Fine-Tuning (SFT)

A base model, while knowledgeable, is not naturally good at following instructions or holding a conversation; it simply continues text in whatever style it was given. Supervised fine-tuning addresses this by training the model further on curated examples of instructions paired with high quality responses, written or reviewed by humans. This step teaches the model the format and style of a helpful assistant.

3.4 Reinforcement Learning from Human Feedback (RLHF)

After supervised fine-tuning, many labs apply reinforcement learning techniques to further align the model's behavior with human preferences. Human reviewers compare pairs of model outputs and indicate which one they prefer; this preference data is used to train a reward model, which then guides further training of the LLM using reinforcement learning algorithms. This process helps make responses more helpful, honest, and safe, and reduces undesirable behaviors.

The reward model itself is a separate neural network trained specifically to predict which of two candidate responses a human would prefer, effectively learning a numerical approximation of human judgment. The main LLM is then updated using a reinforcement learning algorithm, commonly a variant of Proximal Policy Optimization (PPO), which nudges the model's outputs toward responses the reward model scores highly, while a constraint keeps the model from drifting too far from its original, more broadly capable behavior in the process.

3.5 Other Alignment Techniques

In addition to RLHF, researchers use techniques such as Constitutional AI, Direct Preference Optimization (DPO), and rule-based reward systems to refine model behavior. Many of these methods aim to reduce the need for large amounts of human-labeled comparison data while preserving or improving alignment quality.

3.6 Reasoning-Focused Training

A more recent development trains models specifically to produce extended internal reasoning, sometimes called a chain of thought, before producing a final answer. This is reinforced using reward signals tied to whether the final answer is correct, particularly for tasks with verifiable answers such as mathematics, logic, and code. This has substantially improved performance on complex multi-step problems.

3.7 Distillation

Knowledge distillation is a technique where a smaller 'student' model is trained to mimic the outputs or internal behavior of a larger, more capable 'teacher' model. This allows organizations to produce compact models that retain much of the performance of a larger model while being cheaper and faster to run, which is especially valuable for applications with strict latency or cost constraints, such as mobile devices or high-volume customer-facing services.

3.8 Continual and Iterative Training

Rather than treating training as a single one-time event, many organizations now run iterative training cycles: releasing a model, gathering feedback and usage data, and using that information to inform the next round of data curation, fine-tuning, and alignment work for a subsequent version. This iterative loop has become a standard part of how leading LLMs are developed and improved over time.

4. Scale, Parameters, and Emergent Behavior

4.1 What Are Parameters?

Parameters are the internal numerical values (weights) that a neural network adjusts during training to minimize prediction error. A model with more parameters has, in principle, a greater capacity to represent complex patterns in language, though this also requires more data and compute to train effectively, and more memory and computation to run.

4.2 Scaling Laws

Research on scaling laws has shown a fairly predictable relationship between model size, dataset size, compute budget, and resulting performance: as these resources increase together in roughly balanced proportion, the model's loss (a measure of prediction error) decreases in a smooth and predictable way. This insight guided the industry's push toward ever-larger training runs.

One practical implication of scaling laws is that model size and data size need to be increased together, in a roughly balanced way, to get the most benefit from a given compute budget; pouring more compute into an already-oversized model trained on too little data, or vice versa, yields worse results than a more balanced allocation. This insight, often associated with research from labs such as DeepMind and OpenAI in the early 2020s, shifted industry practice away from simply maximizing parameter count toward jointly optimizing model size, dataset size, and training duration for a given compute budget.

4.3 Emergent Abilities

Emergent abilities are capabilities that appear only once a model crosses a certain scale threshold, rather than improving gradually. Examples cited in the research literature include multi-step arithmetic, certain forms of analogical reasoning, and following complex multi-part instructions. The existence and nature of true 'emergence' versus gradual improvement measured with coarse metrics remains a subject of ongoing research debate.

4.4 Diminishing Returns and Efficiency

Simply making models larger is not the only path to better performance. Improvements in data quality, training techniques, architecture efficiency (such as mixture-of-experts designs that activate only a subset of parameters per input), and inference optimization have allowed smaller models to match or exceed the performance of older, larger models, which is important for cost and energy efficiency.

4.5 Mixture-of-Experts (MoE) Architectures

Mixture-of-experts is an architectural pattern in which a model contains many specialized sub-networks, called experts, but only a small subset of them is activated for any given input token, determined by a learned routing mechanism. This allows the total parameter count of the model to be very large, which increases its overall capacity and knowledge, while the actual computation performed per token remains much smaller than a comparably sized dense model would require. This trade-off has made MoE designs popular among several major model families seeking to balance capability against inference cost.

4.6 Model Size Categories in Practice

In practice, providers often offer multiple sizes of a model family, commonly described informally as small, medium, and large, or with names like 'mini,' 'flash,' or 'pro' tiers. Smaller models are cheaper and faster, making them suitable for simple, high-volume tasks such as basic classification or short conversational replies, while larger models are reserved for tasks demanding deeper reasoning, nuanced judgment, or complex multi-step problem solving. Selecting the right size for a given task is an important practical decision that balances quality against latency and cost.

5. Context Windows and Memory

The context window is the maximum amount of text, measured in tokens, that a model can consider at once when generating a response. Anything outside this window is effectively invisible to the model during that particular generation step.

5.1 Why Context Length Matters

A longer context window allows a model to work with longer documents, maintain coherence across long conversations, and reference earlier details accurately. Early transformer-based LLMs were limited to a few thousand tokens; many current models support context windows in the hundreds of thousands of tokens, enabling use cases like analyzing entire books or large codebases in a single pass.

5.2 The Cost of Long Context

Standard self-attention scales quadratically with sequence length, meaning that doubling the context length roughly quadruples the compute and memory required for attention computations. This has motivated research into more efficient attention variants, sliding-window attention, and other architectural tricks that make long-context processing more practical.

5.3 LLMs Do Not Have Persistent Memory by Default

It is important to distinguish a large context window from genuine long-term memory. Most LLMs are stateless between separate conversations: they do not automatically remember previous sessions unless that information is explicitly stored and reintroduced into the context, for example through a memory system, a database, or a retrieval mechanism like RAG.

5.4 Strategies for Working Within Context Limits

When the amount of relevant information exceeds what fits comfortably in a context window, several strategies are commonly used. Summarization condenses earlier parts of a conversation or document into a shorter form that preserves the key points while using fewer tokens. Retrieval, as used in RAG systems, selects only the most relevant pieces of a larger corpus rather than including everything. Sliding-window approaches keep only the most recent portion of a long conversation in context, discarding or summarizing older turns. Choosing the right strategy depends on whether older information needs to be preserved exactly, summarized, or can safely be dropped.

6. Prompting and In-Context Learning

Prompting is the practice of crafting the input text given to an LLM in order to elicit a desired type of response. Because LLMs are highly sensitive to phrasing, structure, and examples provided in the prompt, prompt design has become an important practical skill.

6.1 Zero-Shot, Few-Shot, and In-Context Learning

In zero-shot prompting, the model is asked to perform a task with no examples, relying purely on its pretrained knowledge and instruction-following ability. In few-shot prompting, a handful of example input-output pairs are included directly in the prompt, which often substantially improves accuracy on structured or unfamiliar tasks. This ability to learn a task pattern purely from examples within a single prompt, without any weight updates, is known as in-context learning.

6.2 Chain-of-Thought Prompting

Chain-of-thought prompting encourages the model to write out intermediate reasoning steps before giving a final answer, rather than jumping straight to a conclusion. This technique has been shown to substantially improve performance on arithmetic, logic, and multi-step reasoning tasks, since it gives the model more 'room' to work through a problem incrementally rather than predicting an answer in one shot.

6.3 System Prompts and Role Instructions

Many LLM applications use a system prompt: a special instruction, typically hidden from the end user, that sets the overall behavior, persona, and constraints for the model throughout a conversation. This is distinct from the user's own messages and is generally given higher priority by well-aligned models.

6.4 Practical Prompting Tips

- Be specific about the desired format, length, and tone of the response.
- Provide relevant context and constraints rather than assuming the model can infer them.
- Use examples (few-shot) when the task has a particular structure that is hard to describe in words.

- Break complex tasks into smaller steps, or explicitly ask for step-by-step reasoning.
- Iterate: treat prompting as an experimental process, refining based on the outputs you observe.

6.5 Common Prompting Pitfalls

Several recurring mistakes reduce prompt effectiveness in practice. Overly vague prompts force the model to guess at the user's true intent, often producing a generic response. Conflicting instructions, such as asking for both maximum brevity and exhaustive detail, can confuse the model about which constraint to prioritize. Burying the actual question under excessive unrelated context can cause the model to lose focus on what is actually being asked. Finally, assuming the model remembers something from a much earlier, now-truncated part of a long conversation can lead to responses that seem to ignore information the user believes was already provided.

Technique	Best For	Trade-off
Zero-shot	Simple, well-known tasks	Lower accuracy on unfamiliar formats
Few-shot	Tasks with a specific structure	Uses more context tokens
Chain-of-thought	Multi-step reasoning, math, logic	Longer, slower responses
System prompt	Setting persistent behavior/persona	Hidden from user, needs careful design

7. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique that combines an LLM with an external knowledge source, typically a database of documents that can be searched at query time. Rather than relying solely on knowledge baked into the model's parameters during training, a RAG system retrieves relevant pieces of text and inserts them into the model's context, allowing it to generate answers grounded in that retrieved content.

7.1 Why RAG Matters

- Up-to-date information: The model can use current data without being retrained.
- Reduced hallucination: Grounding answers in retrieved text reduces fabricated or incorrect claims.
- Domain specialization: Organizations can connect an LLM to private or specialized knowledge bases.
- Transparency: Retrieved sources can be cited, allowing users to verify the basis for an answer.
- Cost efficiency: Updating a document index is far cheaper than retraining or fine-tuning a model.

7.2 The RAG Pipeline

A typical RAG pipeline involves several stages. First, source documents are split into smaller chunks of text, since embedding and retrieving entire documents is usually impractical. Each chunk is then converted into a numerical vector, called an embedding, using an embedding model. These vectors are stored in a vector database or vector index that supports efficient similarity search.

When a user submits a query, the query itself is also converted into an embedding. The system then searches the vector index for the chunks whose embeddings are most similar to the query embedding, typically using a similarity measure such as cosine similarity. The most relevant chunks are then inserted into the prompt sent to the LLM, along with the original user query and any system instructions, and the LLM generates its final answer based on this combined context.

7.3 Chunking Strategy

How documents are split into chunks has a major effect on retrieval quality. Chunks that are too large dilute relevance and waste context space; chunks that are too small may lose important surrounding context. Common strategies include fixed-size chunking with overlap between consecutive chunks, splitting along natural boundaries such as paragraphs or sections, and more advanced semantic chunking that groups sentences by topical similarity.

7.4 Building Good Q&A; Pairs for RAG

One increasingly common approach for RAG knowledge bases is to pre-process source material into explicit question-and-answer pairs rather than, or in addition to, raw text chunks. Each answer is self-contained and directly addresses a specific, well-scoped question, which tends to produce more precise retrieval than retrieving arbitrary paragraph chunks, since the question embedding at query time is more likely to closely match the embedding of a similarly phrased stored question.

- Keep each question focused on a single, clear piece of information.
- Write answers that are self-contained and do not depend on surrounding context to make sense.
- Cover the material from multiple angles: definitions, comparisons, examples, and edge cases.
- Use natural phrasing similar to how an actual user might ask the question.
- Avoid overly long answers; concise, accurate answers retrieve and generate more reliably.

7.5 Re-ranking and Hybrid Search

Many production RAG systems add a re-ranking step after initial retrieval, using a more computationally expensive but more accurate model to re-score and reorder the top candidate chunks before they are passed to the LLM. Hybrid search, which combines traditional keyword-based search with vector similarity search, is also widely used to capture both exact term matches and semantic similarity.

7.6 A Worked Example of Q&A-Style; Knowledge Chunks

To illustrate the structure described above, consider source material describing what a context window is. Rather than storing the raw paragraph as a single chunk, it can be reformulated into several targeted question-and-answer pairs, each addressing one clear angle on the same underlying fact: a definition-style question such as 'What is a context window in an LLM?', a consequence-style question such as 'What happens when a conversation exceeds the context window?', and a comparison-style question such as 'What is the difference between context window size and long-term memory?'. Each of these, stored as an independent question-answer pair with a self-contained answer, gives the retrieval system several different, naturally phrased entry points into the same piece of knowledge, increasing the chance that a real user's query will closely match one of them.

7.7 Limitations of RAG

RAG is not a complete solution to every knowledge problem. Retrieval quality depends heavily on how well the knowledge base is curated and chunked; irrelevant or low-quality retrieved chunks can confuse the model or lead it to generate misleading answers. RAG also does not fundamentally change the model's reasoning ability, and very large knowledge bases require careful infrastructure to keep retrieval both fast and accurate.

7.8 Advanced RAG Patterns

Beyond the basic retrieve-then-generate pipeline, several more advanced patterns have become common in production systems. Query rewriting reformulates a user's original question into one or more alternative phrasings before retrieval, which can surface relevant chunks that would not match the original wording closely enough. Multi-hop retrieval performs more than one retrieval step in sequence, using the result of an initial retrieval to inform a follow-up query, which is useful when answering a question requires combining facts found in different parts of a knowledge base. Agentic RAG goes further still, allowing the model itself to decide, step by step, whether to retrieve again, refine its query, or proceed to answer, rather than following a single fixed retrieval step.

7.9 Keeping a RAG Knowledge Base Healthy

A RAG system is only as good as the knowledge base behind it. Practical maintenance includes regularly removing outdated or superseded documents, periodically reviewing which queries return poor or no relevant results, and re-embedding content when the underlying embedding model is upgraded, since embeddings from different model versions are not directly comparable. Establishing a clear update process for who can add or modify entries, and how changes propagate into the live index, helps prevent knowledge bases from silently becoming stale or inconsistent over time.

8. Fine-Tuning Approaches

Fine-tuning refers to further training an existing pretrained model on a smaller, often domain-specific dataset, in order to specialize its behavior. While RAG changes what information a model has access to, fine-tuning changes the model's underlying weights and, therefore, its intrinsic behavior, style, and knowledge.

8.1 Full Fine-Tuning

Full fine-tuning updates all of a model's parameters using new training data. This can be highly effective but requires substantial compute and memory, since gradients and optimizer states must be stored for every parameter, making it impractical for very large models without significant infrastructure.

8.2 Parameter-Efficient Fine-Tuning (PEFT)

To reduce the cost of fine-tuning, parameter-efficient methods update only a small subset of parameters, or introduce a small number of new trainable parameters, while keeping most of the original model frozen. The most widely used technique is LoRA (Low-Rank Adaptation), which injects small trainable low-rank matrices into specific layers of the model. This dramatically reduces the memory and compute needed for fine-tuning while often achieving results close to full fine-tuning.

8.3 When to Fine-Tune vs. Use RAG vs. Prompting

- Use prompting alone for general tasks where the base model's existing knowledge is sufficient.
- Use RAG when the issue is missing or outdated factual knowledge that changes over time.
- Use fine-tuning when you need to change the model's style, format, tone, or specialized skill in a way that is hard to achieve through prompting or retrieval alone.
- These approaches are often combined: a fine-tuned model can still use RAG for up-to-date facts.

8.4 Quantization

Quantization reduces the numerical precision used to store a model's weights, for example converting from 16-bit or 32-bit floating point numbers down to 8-bit or even 4-bit representations. This substantially shrinks the memory footprint required to load and run a model and can speed up inference, usually at a small, carefully managed cost to output quality. Quantization is widely used to make large models practical to run on more modest hardware, including consumer devices.

8.5 Summary Comparison

Approach	Updates Knowledge?	Changes Behavior/Style?	Relative Cost
Prompting	No	Temporarily, per request	Very low

Approach	Updates Knowledge?	Changes Behavior/Style?	Relative Cost
RAG	Yes, via retrieval	No, only adds context	Low to moderate
Fine-tuning (PEFT)	Limited, via training data	Yes, durably	Moderate
Full fine-tuning	Yes, via training data	Yes, durably	High

9. Inference: How LLMs Generate Text

9.1 Autoregressive Generation

Once trained, an LLM generates text autoregressively: it predicts a probability distribution over possible next tokens, selects one token according to some sampling strategy, appends it to the sequence, and repeats the process. This continues until the model produces a special end-of-sequence token or reaches a maximum length limit.

9.2 Sampling Strategies

Several strategies control how a token is chosen from the predicted probability distribution. Greedy decoding always picks the single most likely token, which is deterministic but can produce repetitive or unimaginative text. Temperature sampling adjusts the randomness of selection: lower temperature makes the output more focused and predictable, while higher temperature increases diversity and creativity at the cost of occasional incoherence. Top-k sampling restricts choices to the k most likely tokens, while top-p (nucleus) sampling selects from the smallest set of tokens whose combined probability exceeds a threshold p.

9.3 Inference Cost and Latency

Generating each token requires a full forward pass through the network, making generation inherently sequential and a major source of latency for long outputs. Techniques such as key-value caching (reusing previously computed attention values rather than recomputing them for every new token), speculative decoding, and quantization are widely used to speed up inference and reduce computational cost in production systems.

10. Multimodal LLMs

While early LLMs processed only text, many modern models are multimodal, meaning they can accept and sometimes generate other types of data such as images, audio, and video, in addition to text.

10.1 Vision-Language Models

Vision-language models typically use a separate vision encoder to convert an image into a sequence of embeddings, which are then projected into the same representational space used by the language model and processed alongside text tokens. This allows the model to answer questions about an image, describe its contents, read text within it, or reason about diagrams and charts.

10.2 Audio and Speech

Audio-capable models can transcribe speech to text, understand spoken instructions directly, or generate natural-sounding speech as output, enabling voice assistants and real-time conversational applications without a separate speech-to-text pipeline.

10.3 Why Multimodality Matters

Much of human knowledge and communication is not purely textual. Multimodal capability allows LLMs to be applied to a far broader range of real-world tasks, such as analyzing scientific figures, reviewing user interface designs, interpreting medical images alongside reports, or assisting users through voice in hands-free contexts.

10.4 Document and Visual Reasoning

An increasingly important multimodal capability is reading and reasoning over visually structured documents, such as PDFs, scanned forms, slide decks, and spreadsheets, where layout, tables, and embedded charts carry meaning that plain text extraction would lose. Models capable of processing page images directly, rather than only extracted text, can answer questions that depend on spatial layout, such as identifying which figure a caption refers to or reading values out of a chart.

11. Limitations and Challenges

11.1 Hallucination

Hallucination refers to an LLM generating text that is fluent and confident-sounding but factually incorrect or entirely fabricated. Because LLMs generate text based on learned statistical patterns rather than verified facts retrieved from a trusted source, they can produce plausible-sounding but false information, especially for obscure facts, recent events, or very specific figures.

Hallucination tends to be most common in a few recognizable situations: when a model is asked about something extremely specific or obscure that appeared rarely, if at all, in its training data; when it is asked to cite an exact source, quote, statistic, or citation rather than summarize general knowledge; and when a question contains a false premise that the model fails to recognize and instead builds an answer on top of. Mitigations include grounding responses in retrieved source documents through RAG, asking the model to express uncertainty or cite sources, and, for high-stakes use cases, having a human verify any

specific factual claims before they are relied upon.

11.2 Bias

Because LLMs learn from large collections of human-generated text, they can absorb and reproduce biases present in that data, including stereotypes related to gender, race, culture, or other characteristics. Significant effort goes into auditing and mitigating these biases, but eliminating them entirely remains an open challenge.

Bias can surface in subtle ways, such as a model defaulting to a particular gender when describing an unspecified person in a given profession, producing differing quality or tone of response depending on a name or dialect used in a query, or reflecting the cultural or political assumptions most prevalent in its training data while underrepresenting others. Mitigation strategies include carefully curating and balancing training data, explicit alignment training aimed at fairness and even-handedness, and ongoing evaluation using benchmarks specifically designed to surface biased behavior across demographic groups and topics.

11.3 Reasoning and Math Limitations

Despite impressive performance on many reasoning benchmarks, LLMs can still make basic arithmetic errors, lose track of constraints in long multi-step problems, or produce reasoning that looks coherent but contains subtle logical flaws. Reasoning-focused training and tool use (such as calling a calculator or code interpreter) have substantially improved, but not eliminated, these issues.

A particularly important failure mode is that a model's stated reasoning does not always faithfully reflect the actual process that produced its answer. A model may write out a chain of thought that looks logical and arrive at a correct or incorrect answer somewhat independently of that written reasoning, especially under pressure from a leading or biased question. This means that even detailed, step-by-step explanations from a model should be checked against independent verification for important decisions, rather than trusted purely because the reasoning appears thorough.

11.4 Lack of True Understanding or Grounding

A long-running debate in the field concerns whether LLMs possess genuine understanding of the concepts they discuss, or whether they are sophisticated pattern matchers that produce convincing text without any underlying comprehension. This question has philosophical as well as practical implications for how much LLM output should be trusted without verification.

11.5 Prompt Injection and Security

When LLMs process untrusted external content, such as web pages, documents, or emails, malicious instructions embedded in that content can sometimes manipulate the model into ignoring its original instructions, a vulnerability known as prompt injection. This is an active area of security research, particularly as LLMs are increasingly given the ability to take real-world actions through tools.

11.6 Computational and Environmental Cost

Training and operating large-scale LLMs consumes substantial electricity and specialized hardware, raising both economic and environmental concerns. This has driven significant research into more efficient architectures, smaller specialized models, and better hardware utilization.

11.7 Knowledge Cutoffs and Staleness

Because pretraining uses a fixed snapshot of data collected up to a certain point in time, every LLM has an effective knowledge cutoff beyond which it has no direct training knowledge of new events, products, or developments. Without an external retrieval mechanism such as web search or RAG, a model may confidently describe an outdated state of the world as if it were current, which is one reason retrieval and search tools have become a standard companion to LLM deployments.

11.8 Inconsistency and Sensitivity to Phrasing

LLM outputs can vary noticeably based on small changes in how a question is phrased, the order information is presented, or even random sampling during generation. This sensitivity makes it important to test prompts across many variations in production systems and to avoid over-interpreting a single response as fully representative of a model's general capability on a task.

12. Evaluating LLMs

Assessing how good an LLM actually is requires more than informal impressions; the field uses a combination of automated benchmarks and human evaluation.

12.1 Automated Benchmarks

Standardized benchmarks test models on tasks such as general knowledge question answering, reading comprehension, mathematics, coding, and multi-step reasoning. Examples widely referenced in the literature include MMLU for broad knowledge, HumanEval for code generation, and GSM8K for grade-school-level math word problems, among many others that have evolved over time.

12.2 Human Preference Evaluation

Because many qualities that matter to users, such as helpfulness, tone, and clarity, are difficult to capture with automated metrics alone, human evaluators are often asked to compare two model outputs side by side and indicate which they prefer. Aggregated across many comparisons, this produces preference rankings that better reflect real-world usefulness than benchmark scores alone.

12.3 Limitations of Benchmarks

Benchmarks can become contaminated if test questions leak into training data, inflating scores without reflecting genuine improvement. High benchmark performance also does not guarantee good behavior on tasks outside the benchmark's specific format or domain, which is why real-world evaluation and continuous monitoring remain essential.

12.4 Evaluating RAG Systems Specifically

RAG systems require their own evaluation dimensions beyond generic LLM benchmarks. Retrieval quality can be measured by checking whether the chunks actually retrieved for a given query contain the information needed to answer it correctly. Answer faithfulness checks whether the generated answer is properly grounded in the retrieved content rather than introducing unsupported claims. End-to-end accuracy measures whether the final answer delivered to the user is correct and complete. Tracking these dimensions separately helps diagnose whether a quality problem stems from retrieval, from generation, or from gaps in the underlying knowledge base.

13. AI Agents and Tool Use

An emerging direction in LLM applications involves agents: systems where an LLM does not simply produce a single response, but plans a sequence of actions, calls external tools or APIs, observes the results, and continues reasoning toward a goal across multiple steps.

13.1 Function and Tool Calling

Tool calling allows an LLM to invoke external functions, such as a web search, a calculator, a database query, or a code execution environment, by generating a structured request that a surrounding system executes on its behalf. The results are then returned to the model, which incorporates them into its next response. This extends the model's effective capabilities far beyond what is encoded in its parameters alone.

13.2 Multi-Step Agentic Workflows

In agentic workflows, an LLM may break a complex task into sub-tasks, decide which tools to use for each, execute them in sequence, evaluate intermediate results, and adjust its plan if something goes wrong. Applications include coding agents that write, run, and debug software; research agents that gather and synthesize information from multiple sources; and operational agents that interact with business software on a user's behalf.

13.3 Challenges in Agentic Systems

Agentic systems introduce new risks beyond those of single-turn LLM use, including compounding errors across multiple steps, the security risks of executing real-world actions, and the difficulty of ensuring an agent reliably stays within its intended scope, especially when interacting with untrusted external content.

13.4 Coding Agents

Coding agents are a particularly mature category of agentic system, capable of reading an existing codebase, planning a set of changes needed to implement a feature or fix a bug, writing the necessary code, running tests, interpreting any failures, and iterating until the task is complete. These agents typically operate inside a sandboxed environment with access to a file system, a terminal, and version control, and they have become a significant productivity tool for professional software development.

13.5 Orchestration and Multi-Agent Systems

Some applications use multiple specialized agents that collaborate on a task, with one agent acting as an orchestrator that delegates sub-tasks to other agents better suited to specific parts of the problem, such as one agent specialized in research and another in writing or coding. While this can improve performance on complex tasks by allowing each agent to focus on a narrower responsibility, it also introduces additional coordination overhead and potential points of failure, so simpler single-agent designs are often preferred unless task complexity clearly justifies the added architecture.

14. Real-World Applications of LLMs

14.1 Software Development

LLMs assist with writing, explaining, reviewing, and debugging code, generating tests, and even autonomously implementing features within larger coding agent workflows, substantially changing how software is built.

- Code generation: producing functions, scripts, or boilerplate from a natural language description.
- Code review: flagging bugs, security issues, or style inconsistencies in proposed changes.
- Debugging: explaining error messages and suggesting likely root causes and fixes.
- Documentation: generating comments, docstrings, and explanatory README content from existing code.
- Test generation: producing unit tests that exercise a function's expected behavior and edge cases.

14.2 Customer Support and Business Operations

Many organizations use LLMs to power chatbots and virtual assistants that handle customer inquiries, draft email responses, summarize support tickets, and route complex issues to human agents, often combined with RAG over internal documentation.

14.3 Education and Tutoring

LLMs can explain concepts at varying levels of depth, generate practice problems, provide feedback on writing, and adapt explanations to a learner's apparent level of understanding, functioning as an on-demand tutor across many subjects.

14.4 Research and Knowledge Work

Researchers and analysts use LLMs to summarize papers, extract structured data from unstructured documents, brainstorm hypotheses, and draft reports, accelerating many stages of the research and writing process.

A particularly common pattern combines an LLM with web search or a private RAG knowledge base to produce research summaries grounded in current, retrievable sources rather than relying solely on the model's pretrained knowledge, which may be outdated or insufficiently detailed for a specialized domain. This pairing allows knowledge workers to ask open-ended questions and receive synthesized answers that cite and reflect the actual underlying source material.

14.5 Creative Work

LLMs assist with brainstorming, drafting, and editing in creative writing, marketing copy, and content production, generally functioning best as a collaborative tool alongside human creativity and judgment rather than a fully autonomous replacement.

14.6 Healthcare and Scientific Domains

Specialized and general LLMs are being explored for summarizing clinical notes, assisting with literature review, and supporting research workflows in fields like biology and chemistry, though high-stakes domains like healthcare require careful validation, regulatory compliance, and human oversight before deployment.

15. Cost, Deployment, and Security Considerations

Beyond model quality, practical LLM adoption depends heavily on cost, deployment architecture, and security practices. These operational concerns often determine whether an LLM-based application succeeds in production.

15.1 Understanding API Pricing

Most hosted LLM providers charge based on the number of tokens processed, typically with separate rates for input tokens (the prompt and any retrieved context) and output tokens (the generated response), since generating output is generally more computationally expensive than reading input. Longer conversations, large retrieved documents, and verbose system prompts all increase input token counts, while longer or more detailed responses increase output token counts. Understanding this cost structure is essential when designing RAG pipelines, since the number and size of retrieved chunks directly affects cost per query.

15.2 Deployment Models

Organizations can access LLMs through fully managed cloud APIs, where the provider handles all infrastructure; through private cloud deployments, where a model runs on dedicated cloud resources the organization controls; or through fully local, on-premises deployment of an open-weight model, which offers maximum control over data residency at the cost of having to manage the necessary hardware and operational expertise. The right choice generally depends on data sensitivity, scale, latency requirements, and available technical resources.

15.3 Data Privacy and Confidentiality

When sensitive or proprietary information is sent to an LLM, whether directly or as retrieved context in a RAG system, organizations need to understand the provider's data retention and usage policies, including whether submitted data may be used for further model training. Many providers offer specific data-handling commitments or zero-retention options for business and enterprise customers, and these terms should be reviewed carefully before sending confidential data.

15.4 Access Control in RAG Systems

A subtle but important security consideration in RAG deployments is ensuring that retrieval respects the same access permissions as the underlying documents. If a knowledge base contains documents with different confidentiality levels, the retrieval layer must filter results according to the requesting user's permissions, or sensitive information could be surfaced to users who should not have access to it, even if the underlying document store itself is properly access-controlled.

15.5 Monitoring and Observability

Production LLM systems benefit from logging and monitoring of inputs, outputs, retrieved context, latency, and cost, both to catch quality regressions and to detect misuse or unexpected behavior. For RAG systems specifically, tracking which documents are retrieved most and least often can reveal gaps in the knowledge base or chunking issues that hurt answer quality.

15.6 Guarding Against Prompt Injection in Retrieved Content

Because RAG systems insert externally sourced text directly into a model's context, retrieved documents that contain malicious or manipulative instructions can pose a prompt injection risk just as untrusted web pages or emails can. Treating retrieved content as data rather than as instructions, clearly separating it from system and user instructions within the prompt structure, and restricting what actions a model can take based purely on retrieved text, are all important defensive practices for RAG systems that ingest content from less trusted sources.

16. The Future of LLMs

The field continues to evolve rapidly. While it is impossible to predict every development with certainty, several broad directions appear likely to remain important.

- Improved reasoning: Continued progress on multi-step reasoning, planning, and reliability.
- Greater efficiency: Smaller, cheaper models that match the performance of much larger predecessors.
- Longer and smarter memory: Better mechanisms for retaining and using information across long interactions and across sessions, beyond simply extending the raw context window.
- Deeper multimodality: More seamless integration of text, image, audio, and video understanding and generation.
- Greater agentic capability: More reliable systems that can plan and execute complex, multi-step real-world tasks with appropriate human oversight.
- Continued focus on safety and alignment: Ongoing research into making models more honest, controllable, and robust against misuse as their capabilities and real-world deployment grow.

Understanding the fundamentals covered in this guide, the transformer architecture, training pipeline, scaling behavior, prompting techniques, retrieval augmentation, and known limitations, provides a solid foundation for following these developments as the field continues to mature.

17. Glossary of Key Terms

This glossary collects the key terms introduced throughout this guide for quick reference.

- Token: A unit of text (word, sub-word, or character) that an LLM processes as a discrete input unit.
- Embedding: A dense numerical vector representation of a token, word, sentence, or document.
- Transformer: The neural network architecture based on self-attention that underlies most modern LLMs.

- Self-attention: A mechanism that lets a model weigh the relevance of every token relative to every other token in a sequence.
- Parameter: A trainable numerical weight inside a neural network, adjusted during training.
- Context window: The maximum number of tokens a model can consider at once during generation.
- Pretraining: The initial, large-scale training phase where a model learns from massive text corpora.
- Base model / foundation model: The result of pretraining, before instruction tuning or alignment.
- Fine-tuning: Further training a pretrained model on a smaller, more specific dataset.
- Supervised fine-tuning (SFT): Fine-tuning on curated instruction-response examples to teach assistant behavior.
- RLHF: Reinforcement Learning from Human Feedback, used to align model behavior with human preferences.
- LoRA: Low-Rank Adaptation, a parameter-efficient fine-tuning technique.
- Quantization: Reducing the numerical precision of model weights to save memory and increase speed.
- Hallucination: Confident but factually incorrect or fabricated output generated by an LLM.
- RAG: Retrieval-Augmented Generation, combining an LLM with retrieval over an external knowledge base.
- Vector database: A database optimized for storing and searching high-dimensional embedding vectors.
- Chunking: Splitting source documents into smaller pieces of text suitable for embedding and retrieval.
- Re-ranking: Re-scoring an initial set of retrieved results with a more precise model before final use.
- Chain of thought: Step-by-step intermediate reasoning produced by a model before its final answer.
- Prompt: The input text given to an LLM to elicit a desired response.
- System prompt: A hidden, persistent instruction that sets a model's overall behavior for a session.
- Few-shot prompting: Including example input-output pairs directly in a prompt to guide a model's behavior.
- In-context learning: A model's ability to learn a task pattern from examples within a single prompt, without updating its weights.
- Inference: The process of running a trained model to generate a prediction or response.
- Sampling temperature: A parameter controlling the randomness of token selection during generation.

- Mixture-of-experts (MoE): An architecture that activates only a subset of specialized sub-networks per input.
- Multimodal model: A model capable of processing or generating more than one type of data, such as text and images.
- Agent: An LLM-based system that plans and executes multi-step actions, often using external tools.
- Tool calling / function calling: A model's ability to invoke external functions or APIs as part of generating a response.
- Knowledge cutoff: The point in time after which a model has no direct training knowledge of events.
- Prompt injection: A security vulnerability where malicious instructions embedded in external content manipulate a model's behavior.